

Design and Analysis of Algorithms

Module 5: Trees

Slides © Grant Williams, [CC BY-SA 4.0](#).

This is an open educational resource.

Feel free to submit fixes, improvements, and new material [here](#).

Introduction

In this module we will review trees.

We will learn about their benefits, their drawbacks, and how to mitigate them.

However, before we get too much into it, I have a question:

What is the point? Why do we care about trees?

They feature prominently in CS education, but how often have you been like "I need a binary tree for this" when doing your software engineering or capstone projects?

The point of trees?

Trees are *one* way of implementing dictionaries (aka *maps*).

Hashmaps are another.

These two completely different approaches both allow us to create the same abstract structure. And it turns out, there are reasons to use both.

[So what is a dictionary, and how does it work?]

Dictionaries

A dictionary can be seen either as a generalization of an array, or as a special case of relation (partial map).

Ordinarily, arrays are a kind of function. You give them an index, and they give you some kind of value.

We say that they are *partial* functions, because arrays are not required to have a value for every index. Some indices might not have a value. If an array has size 100, then any index ≥ 100 is not going to be there.

However, array indices are limited to natural numbers (usually, technically in C you can access arrays with negative indices for extremely arcane purposes). And the keys need to be adjacent for efficiency reasons.

Dictionaries (2)

Dictionaries are data structures that break those two limitations:

1. You can associate (almost) any kind of key with any kind of value.
2. Integer keys don't need to be adjacent. You can have 1: "Hello" and 1000000000: "Goodbye" in the same data structure without allocating a one billion and one cell array.

[So, how do we implement them?]

Default: hashmaps

By default, dictionaries are almost always just a thin wrapper over a hashmap.

That's how it works in Python, Lua, Ruby, and many other languages.

Hashmaps are fast, cache efficient, and typically waste no more than twice the amount of memory necessary when they are reasonably large.

However, Hashmaps have a major limitation [what is it?]

Hashmaps are not sorted

The whole point of a hash function is that it be as close to uniform random as possible.

That is, the hashvalue of a particular input should be as unpredictable as possible without evaluating the function.

Why? Because we want to use the whole hashtable. If the function frequently returns some keys more than others, we end up with more collisions.

So it's not only unsorted: it's *super* unsorted. It's as unsorted as possible.

On the other hand...

Trees are sorted

Binary search trees in particular are sorted.

Specifically, an in-order traversal will be in order.

[Why?]

Trees are sorted (2)

By induction on a tree:

- If it's empty, it's sorted
- If in-order traversal of its child nodes is sorted, and everything in left < root < everything in right, then `inorder(left) ++ root ++ inorder(right)` is sorted.

So the normal property of BSTs, that the node's value is always between the values of its left and right children, naturally makes it so that in-order traversal is sorted.

Questions?

Who cares?

I have a question: why do we need dictionaries to be sorted?

It seems like every time we use a dictionary it's because we know the keys.

And even if we don't, like, if we want to iterate over all of them, we can just put the keys in an array. That still seems better than using a tree.

Good examples

```
SELECT * FROM Transactions WHERE TxDate BETWEEN  
DATEADD(month, -1, CURRENT_DATE) AND CURRENT_DATE;
```

We're choosing all transactions in the last month.

How does the database actually do that?

One way to do it: put the dates in a tree.

The key is the date, the value (aka satellite data, we'll talk about it soon) is the row.

Good examples (2)

The database can quickly scan over the data tree, in order, to find the start and end.

This ends up being $O(\lg n + k)$:

- $\lg n$ for the initial search in the BST
- k for the number of elements in the range

But it needs to stay sorted. We don't want to have inserts take too long.

Tree dictionaries

So, sometimes we need a structure that stays sorted, but also can implement dictionaries.

How do we implement a dictionary with a tree?

Let's review the BST data structure to make sure we understand it, and then implement a simple dictionary.

BstNode

```
typedef struct bst_node_t {  
    struct bst_node_t *left, *right;  
    void* data;  
} BstNode;
```

You've seen it before, but maybe not in C.

Remember: the typedef only works outside the struct, so we need to scope our `bst_node_t` pointer with "struct".

The data

What is `data` ?

It's whatever you want it to be. That's why it's a `void*`.

It could be a string, a tuple, or even an integer casted to a pointer.

When you first learned about BSTs, likely the data was hardcoded to be an `int` or a string. But think back to the database example: a database row is a lot more complicated.

The comparator

First, to make a BST, we need a comparator. A comparator is something that compares two data elements and tells you which one is smaller.

```
static inline int node_compare_int_default(void* a, void* b) {  
    return (intptr_t)a - (intptr_t)b;  
}
```

This is a simple comparator that assumes the void* aren't actually pointing to anything useful, and instead they are just reinterpreted as integers (`intptr_t` is guaranteed to be as large as a pointer, but it's signed).

It's important to decide when you're going to do this, vs have the pointer actually point to something. For db-rows we would definitely do the latter:

```
return ((SomeRow*)a)->date - ((SomeRow*)b)->date ;
```

Function pointer reminder

In an OO language the comparator could be an object or a class.

In this case it is a function. It returns < 0 if a is less than b . 0 if equal. > 0 otherwise.

The type looks like this:

```
int (*)(void* a, void* b)
```

That's a function pointer. And we never, ever use them without a typedef.

Function pointer typedef

```
typedef int (*NodeComparer)(void* a, void* b);
```

Now the pointer has a name: it's a `NodeComparer` .

How is it used? We create a field of this type in our tree structure, and call it every time we do an insertion or a lookup.

Speaking of the tree structure...

The struct

```
typedef struct bst_t {  
    Pool pool;  
    size_t size;  
    BstNode* root;  
    NodeComparer comparer;  
} Bst;
```

The root and size are self-explanatory. The comparer is an instance of that typedef. What about the pool?

It's the pool allocator we saw in the first extra lecture.

[Why are we using the pool?]

The pool

If we didn't use the pool, over time, through insertions and deletions, the tree would become scattered. This is worse for cache locality.

That said, our pool pages are still in a linked list, and trees are non-linear data structures, so it's not like they're guaranteed to be adjacent.

However, we can free the entire tree with a single `pool_destroy` call.

It's not absolutely required to use a pool. Feel free to use `malloc` and `free`. There are other benefits, too,

Questions?

Inserting into the tree

You've already seen a simple insertion method for binary search trees.

Briefly here is a review of the pseudo-code:

```
void insert(BstNode** node, value) {  
    if (!*node)  
        *node = create a new node with value  
    else if (value < (*node)->value)  
        insert ((*node)->left, value);  
    else if (value == (*node)->value)  
        *node = deallocate the old value and replace  
    else  
        insert ((*node)->right, value);  
}
```

Here, we're using a double pointer. Why? [Ideas?]

The double pointer

We have two fundamentally different behaviors to handle:

- The tree is empty and root is null
- The tree has a root

If we insert to a null node, we want to fill it in with an actual node.

If there is a node there, we want to do normal insert things (go left or right depending on the result of the comparison)

The double pointer (2)

If we used a single pointer:

```
void insert(BstNode* node, value) {  
    if (!node)  
        node = create a new node with value;  
    // ^^ doesn't do anything  
    ...  
}
```

Would leak memory, because we'd be changing the pointer which is a parameter to the function, but the caller of the function would not be aware of it.

The double pointer (3)

Alternatively, we could return the new pointer...

```
BstNode* insert(BstNode* node, value) {  
    if (!node)  
        return (create a new node with value);  
    ...  
}
```

But then the user of the function would have to check if the root were changed, and then assign the result to the root of the tree. A more complicated design.

The double pointer (4)

In the same way that a single pointer can be used to modify a non-pointer variable from a function, a double pointer can be used to modify a pointer.

So we can pass the *address* of the root pointer (not the root pointer itself), and then it will be overwritten if the root changes.

Does this make sense? We have time for questions if we need to cover double pointers in more detail.

Questions?

Practice

- Recall the tree lookup function you learned in Algorithms and Data Structures. Can you implement it using our Bst structure?
- Does it require a double pointer?
- How would you prove that tree insertions are $\Theta(\lg n)$ on a balanced tree?
- How would you prove that tree insertions are $\Theta(n)$ in the worst case?
- What is the best case Θ for an insertion?

The drawbacks of BSTs

BSTs are useful, but they have a huge drawback.

[what is it?]

They can become imbalanced

Some BSTs are good and some are bad.

If we insert 10, 5, and 15, in that order, we have a balanced BST.

If we insert 5, 10, and 15, in that order, *or the reverse*, we have an imbalanced BST.

[What's the problem with imbalanced BSTs?]

Imbalanced trees become linked lists

The most perfectly imbalanced tree is a linked list with extra pointers.

So it's not even a *good* linked list.

Unfortunately, we can't reasonably say "hey make sure your data is random". We don't know the data in advance.

We need a lower bound on insertion and lookup time, but to get that, we need to modify the tree.

First approaches

The earliest approach was to just to rotate the tree if it gets even a little out of height-balance. This approach was called the AVL tree.

The AVL tree was named after the Latinized names of its inventors: Georgy Adelson-Velsky and Evgenii Mikhailovich Landis

AVL trees

The basic operation is simple:

- Every node tracks either its height, or the height difference between its children.
- This difference is called the "balance". It must always be within $-1 \leq b \leq 1$
- If inserting causes an imbalance, we rotate the node at which there is an imbalance.

AVL trees example

For example, if we insert 1, then 2, then 3, an ordinary BST would look like this:

$1 \rightarrow 2 \rightarrow 3$

This is imbalanced, because 1, has a right-child height of 2, but a left-child height of 0. So its balance is 2, which is outside the legal range.

A BST would then rotate the root to the left to obtain:

$1 \leftarrow 2 \rightarrow 3$

[We can do more examples on the whiteboard]

AVL tree properties

AVL trees are very tightly balanced. Even the slightest bit of imbalance will force a rotation.

Deleting is more expensive: sometimes you have to rotate many times after a delete.

However, these downsides come with an upside: a tightly balanced tree is fast to do lookups on.

Other self-balancing BSTs

There are other kinds of self-balancing BSTs. For example, red-black trees.

Red-black trees don't have as strict height balance requirements that AVL trees do. Instead, one side won't be twice as deep as the other.

This ends up being a tradeoff. Insertions are typically a bit cheaper, because they are less likely to result in rotations. However, the tree isn't kept in as good of a condition.

Questions?

Why choose?

So, we really have a choice:

- More balance means faster lookups, but expensive inserts and deletes.
- Less balance means slower lookups, and eventually it can get so bad that inserts will start to slow down too.

Who says that the right amount of balance is to be between $-1 \leq b \leq 1$? Who says it's within a 2/1 ratio? What if the perfect amount of balance for a given problem is to have no more than exactly 72% of nodes on one side of the tree?

This is where we get to scapegoat trees.

Scapegoat Trees

Scapegoat trees are a kind of self-balancing BST which let you configure exactly how out of whack you want the tree to be allowed to get before it rebalances.

Rebalancing a scapegoat tree is very expensive: it rebalances *perfectly*. You will get the theoretically most balanced tree you can get with that data.

However, you don't have to do it very often, and you also can pick how *much* you rebalance. You typically choose a small portion of the tree to rebalance.

Scapegoat Trees (2)

Scapegoat trees are actually required by the master syllabus for this class. I have to assign a project requiring them.

Also, according to the master syllabus, you are required to read the papers in which they were introduced yourself and implement the data structure.

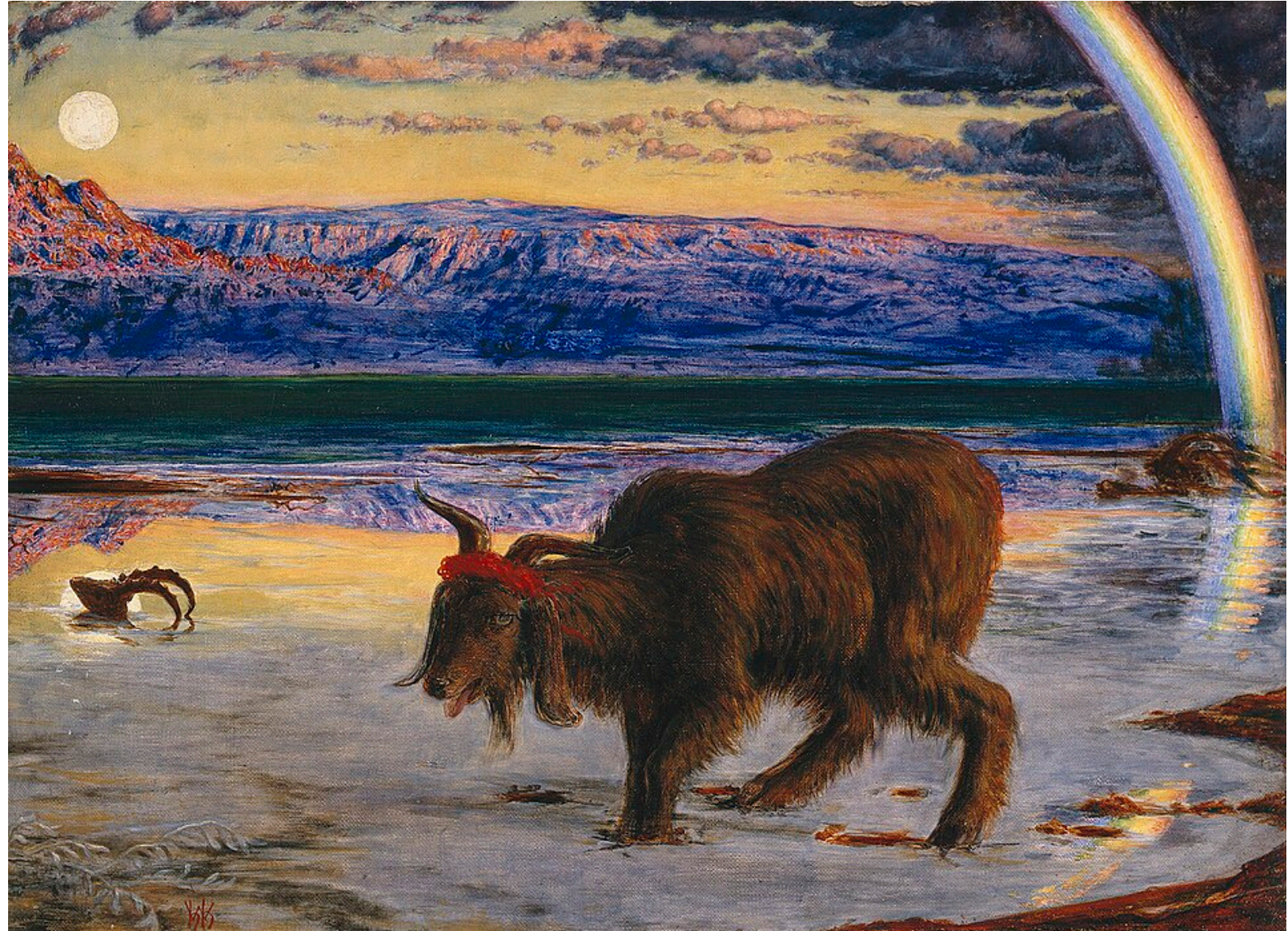
That's a little overwhelming, so I'm going to give you a high-level understanding. I won't provide code, but the following slides should give you a better understanding so that when you do read the papers, it will be a bit more familiar.

What even is a scapegoat?

The scapegoat

Scapegoat rituals are found in many cultures around the world.

An animal (e.g., a goat) is symbolically burdened with the sins of the community, and cast out.



The scapegoat (2)

In our modern society, we only do this figuratively. A scapegoat is someone we blame for something going wrong so that we don't have to address the real causes, and can instead just kick the person out and keep everything the same.

When it comes to self-balancing trees, we can't really change the underlying process. Data is going to come in in some kind of order, and we have to keep it sorted.

But we can blame a sub-tree. If we say "hey, this tree is out of whack and it screwed up the insert", we can then replace that sub-tree with a perfectly balanced one.

Scapegoat trees

Scapegoat trees follow this principle.

We decide in advance how imbalanced we want them to be allowed to be.

If inserting into the tree breaks that invariant for the whole tree, we search for the deepest node for which it is broken. Then we blame that node for causing the problem, and rebuild the sub-tree perfectly balanced at that node.

So how do we represent balance?

α balance

We represent max-imbalance, called α , as the largest proportion of the tree which is allowed on either side.

So if $\alpha = 75\%$, that means that the largest child of the root may have up to 75% of the total amount of nodes. We call such a tree 75% weight-balanced.

Weight balance refers to the total size of the trees on either side. As opposed to height balance, which is based on their height.

If the tree has 1000 nodes, then, with an α of 75%, the root's largest child can have up to 750. If it has more, then the tree is no longer α -weight-balanced.

Technically, we allow 1 node of give. So the larger child needs to have *more than* 75% + 1 nodes before we do anything. We'll explain why later.

Knowledge check

- What α would we pick if we wanted the tree to be perfectly balanced?
- What α would we pick if we never wanted the tree to be re-balanced?
- What is the meaningful range of values that α can inhabit? Does an α of 2 make sense? What about an alpha of 25%?

α tradeoffs: knowledge check answers

This system allows us to decide how often we want to rebalance.

- If we want a perfectly balanced tree, we want $\alpha = 50\%$. That means that the "larger" child must be no more than half the size of the total tree. But that means the other child has the other half. So they're 50/50.
(In practice, perfectly balanced trees only happen at powers of 2, so we allow a slight amount of imbalance even with $\alpha = 50\%$)
- If we want to turn off re-balancing, we pick $\alpha = 100\%$. Now we allow the largest child to have up to all the nodes. This will always be the case.
- The meaningful range is $\alpha \in [.5, 1.0]$ $\alpha < .5$ is contradictory: how can the *larger* child have less than half the tree? Then the *smaller* child has more than half?
 $\alpha > 1.0$ is redundant. The larger child can have *more* than all the nodes?

Questions?

Detecting imbalance

Consider this goofy tree:

$1 \leftarrow 2 \leftarrow 3 \leftarrow 4 \leftarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9$

This tree's root is technically both weight-balanced and height-balanced:

- The left child of root has the same number of nodes as the right child (4 each).
- They also both have the same height (4 each).

And yet, this tree itself is not balanced. The problem is that while the root is balanced, the left and right children of the root (the 4 and 6) are not balanced. Each is, in fact, perfectly imbalanced in opposite directions.

We'd like to be able to detect this as it happens. After an insert, we'd like to know that part of the tree has become imbalanced.

α -height

It turns out, knowing the α -weight tells us how high a sub-tree can be. The max height for a given α -weight is called α -height.

How do we compute the α -height?

Imagine that $\alpha = 2/3$. We'd like that no more than two-thirds of the nodes be on one side. What does that say about the maximum depth we can achieve?

α -height (2)

Every time we go down a level:

$$n_{\text{child}} \leq (2/3) \cdot n_{\text{parent}}$$

The $\alpha = 2/3$ tells us that the child must be no more than two-thirds of the parent. Otherwise, we've violated the balance.

What about going down two levels?

$$n_{\text{child}} \leq (2/3)^2 \cdot n_{\text{parent}}$$

α -height (3)

What is the absolute deepest we can go and have the relation still hold?

We don't know, that's what we're trying to solve for.

However, we do know for certain that the size of the child at that point will be 1.

Why?

Because we check whenever we insert a node. When we insert a node, it is the root of a tree of size 1. So after inserting a node, this inequality holds:

$$1 \leq (2/3)^d \cdot n$$

Where d is the depth of the insertion, and n is the number of nodes in the tree.

α -height (4)

$$1 \leq (2/3)^d \cdot n \equiv$$

$$1/n \leq (2/3)^d \equiv$$

$$n \geq (3/2)^d \equiv \leftarrow \text{we take the reciprocal, so the inequality flips}$$

$$\log_{(3/2)}(n) \geq \log_{(3/2)}((3/2)^d) \equiv$$

$$\log_{(3/2)}(n) \geq d \leftarrow \text{we use the log to solve for the exponent}$$

So we solved for the maximum depth we can allow. As long as the depth of an insertion is less than $\log_{(1/\alpha)}(n)$, where n is the number of nodes in the tree, we're balanced.

In practice, we are okay with d being one greater, because in cases where we have $\alpha = 50\%$, we don't want it to be impossible for a tree with 4 nodes to ever be balanced. [whiteboard]

The scapegoat tree rule

So this is the rule:

- When we insert into a scapegoat tree, we track how far down we are. Every recursive call increases the depth by 1.
- The balance condition is: $d \leq \log_{(1/\alpha)}(n) + 1$.
- If that inequality holds, we do nothing.
- Otherwise, we need to find a scapegoat...

But first...

Questions?

Finding the scapegoat

Consider this imbalanced tree rooted at 2:

$$1 \leftarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

Suppose that we've chosen $\alpha = 0.6$.

First, let's compute $1/\alpha = 1/0.6 \approx 1.67$

What is the maximum depth we can permit with this tree?

$$d \leq \lfloor \log_{1.67}(5) \rfloor + 1 \equiv d \leq \lfloor \log(5) / \log(1.67) \rfloor + 1 \equiv d \leq 3 + 1 \equiv d \leq 4$$

So if $d > 4$, we need to do something.

(I'm taking the floor because the depth is an integer, so we can safely round down.)

Finding the scapegoat (2)

Let's insert 6. This is our new tree

$$1 \leftarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$$

And what's the depth of that insertion? $d = 5$. Uh-oh, now we need a scapegoat. (I'm treating depth as the height of the tree. So the root has depth 1.)

To do this, we start at the 6, and work our way up, parent by parent. At each point, we evaluate this inequality:

$$i \leq \log(n_{\text{subtree}}) / \log(1/\alpha) + 1$$

Where i is the height of the tree at each point. i starts at 1, and goes up by 1 for each parent. If this inequality ever fails, we have found our scapegoat.

Finding the scapegoat (3)

Let's do this step-by-step.

We just inserted 6. Is that one node imbalanced?

$$1 \leq \log(1) / \log(1.67) + 1 = 0 + 1 = 1$$

Nope, it's fine, let's keep going.

The parent is 5. Is that node imbalanced?

$$2 \leq \lfloor \log(2) / \log(1.67) \rfloor + 1 = 1 + 1 = 2$$

Yup, $2 \leq 2$, let's keep going.

Finding the scapegoat (4)

Notice that i is increasing for each parent we consider. And the size is also increasing by 1. But if the tree were not degenerate, the size would increase by more.

The grandparent is 4. Is that node imbalanced?

$$3 \leq \lfloor \log(3) / \log(1.67) \rfloor + 1 = 3$$

Just barely balanced (within 1).

What about 3?

$$4 \leq \lfloor \log(4) / \log(1.67) \rfloor + 1 = 3$$

Uh oh.

Finding the scapegoat (5)

$1 \leftarrow 2 \rightarrow \mathbf{3} \rightarrow 4 \rightarrow 5 \rightarrow 6$

Can you *believe* this 3? It's completely imbalanced! If it weren't for them, everything would have been fine. It's not 2's fault!

3, we have some concerns we need to talk about.

What now?

So we have someone to blame. Now what?

Now, we rebuild the tree centered at 3.

What does it mean to rebuild the tree?

Here are the nodes in the sub-tree, in order:

3, 4, 5, 6

Choose the median, round down. That's the new root. Do the same to the left and right.

We get this new sub-tree: $3 \leftarrow 4 \rightarrow 5 \rightarrow 6$

And this is inserted back into the whole tree:

$1 \leftarrow 2 \rightarrow (3 \leftarrow 4 \rightarrow 5 \rightarrow 6)$

(2 is the root of the whole tree, and 4 is the root of its right child)

Questions?

More on rebuilding trees

In practice, we don't need to compute the log each time. Which is good because that's an expensive operation.

Instead, we can just directly use the size of the ancestor.

If we rebuild the first ancestor i for which $\text{size}(i) > \alpha \cdot \text{size}(\text{parent})$, that is equivalent.

Why is it equivalent? Because if the tree is balanced, that should never happen. No node should be larger than the alpha fraction of its parent.

More algebra follows...

Finding the scapegoat quickly

Remember that we were searching for the first i^{th} ancestor of the inserted node, such that $i > \log_{(1/\alpha)}(\text{size}(i))$

Raise both sides by $1/\alpha$:

$$(1/\alpha)^i > \text{size}(i)$$

This is the first i we find moving up from the inserted node with this inequality. The previous i did not have this property (including $i = 1$):

$$(1/\alpha)^{i-1} \leq \text{size}(i-1)$$

Finding the scapegoat quickly (2)

We can divide those two inequalities.

$(1/\alpha)^i > \text{size}(i)$ means that if we divide the right by a larger value than the left, the inequality will still be true. Therefore.

$$\frac{(1/\alpha)^i}{(1/\alpha)^{i-1}} > \frac{\text{size}(i)}{\text{size}(i-1)}$$

After simplifying:

$$(1/\alpha) > \frac{\text{size}(i)}{\text{size}(i-1)} \equiv \alpha < \frac{\text{size}(i-1)}{\text{size}(i)} \equiv \alpha \cdot \text{size}(i) < \text{size}(i-1)$$

Remember here that i is the parent, and $i - 1$ is the child. So this is how we derive that inequality from before.

In other words, go back up from your insertion until you find a child that is bigger than alpha times its parent.

Go back up from the insertion?

What do we mean by "go back up from the insertion"? How do you do that?

Scapegoat trees were envisioned to not require a parent pointer in their struct. They were designed to use ordinary binary search tree structures.

Why? Avoiding the parent pointer improves the cache efficiency and memory overhead of the tree.

As a result, they wanted to build a self-balancing tree that did not require parent pointers.

Now, AVL trees also don't technically require them, but it ends up being really annoying to delete from them without parent pointers. Same with red-black trees.

Go back up from the insertion? (2)

So, as you insert down into the tree, you are expected to maintain a list of ancestors.

How big should the list be? For any reasonable value of alpha you can use a static list. Unless the tree is *really* imbalanced it shouldn't need more than 1000 depth.

Feel free to use an array list of your own making though. Double the array list every time it gets full and you will have amortized $O(1)$ insertions.

Go back up from the insertion? (3)

Then, after inserting, the length of that list (+ 1) is the depth. If that number is larger than the alpha depth of the tree, we have a problem.

Whom do we blame? Iterate over the list of ancestors in reverse order. If we find an ancestor i where $\alpha \cdot \text{size}(i - 1) < \text{size}(i)$, we make i the scapegoat.

(note: this formula is slightly different from the previous one, because here, i is the index in an array and $i = 0$ is the root, whereas before, $i = 1$ was the node we just inserted)

Eventually, we *will* reach a scapegoat. The root can be a scapegoat too.

Getting the size

But how do we get the size on the way up?

In the original paper, the authors wanted the tree node struct to be small, so it doesn't store the size of the node.

Instead, we start at a size of 1. We store that in an accumulator. Then, whenever we look at the parent node, we recursively compute the size of the sibling, + 1 for the parent, and add those values to the accumulator. We end up needing to add potentially all the nodes in the tree.

This is $O(n)$, so why doesn't that destroy the performance of the tree?

Amortized analysis

Remember how we did best-case, worst-case, and average-case analysis?

There's another kind, called amortized analysis.

Sometimes you will hear of a data structure having "amortized $\Theta(n)$ ". A good example of that is array lists.

Array lists store an array, say, with a capacity of 16. They also store the current size, which let's say is 10.

Amortized analysis (2)

When you add something to the array, it's normally $\Theta(1)$. You just put it at the end and increment the size.

However, if you keep doing this, eventually the array will fill up. Then what?

Then you allocate a new array that is twice as big, and copy over the values. This costs $\Theta(n)$ to copy everything over.

So does that mean that adding to the end of an array list is $O(n)$? That's bad, though.

Sort of...

Amortized analysis (3)

Yes, in the worst case, adding to the array is very expensive.

Let's say that it costs 1 time unit to copy one element.

If it starts with capacity 1, it takes 1 time unit to fill it up, and 1 time unit to copy it over.
Now it has capacity 2.

It takes 1 more time unit to fill it up. 2 time units to copy it. Now capacity = 4.

It takes 2 more units to fill, 4 more to copy. Capacity = 8...

let's make a table...

Amortized analysis (4)

Capacity	Time to fill from previous capacity	Time to reallocate
1	1	1
2	1	2
4	2	4
8	4	8
16	8	16
32	16	32

See the pattern?

Amortized analysis (5)

Everytime we reallocate the array, we have twice as long until we need to reallocate.

The future reallocation time is twice the time it takes to fill up half the array.

So we can just pretend that it takes 3 time units to append one element.

For each element between size 8 and 16, we pay one time unit to append that element.

However, in the future, will have to pay 16 time units. So total = $(16-8) + 16 = 24 = 3 \cdot 8$

For each element between size 16 and 32, we pay 1 unit to append. However, in the future we will have to pay 32. Total = $(32 - 16) + 32 = 48 = 3 \cdot 16$

We always pay a total of three time units per element, even though we keep having to copy more and more

Amortized analysis (6), why?

The reason: every time we allocate, we pay twice as much. But...

Every time we allocate, we go twice as long before we need to re-allocate.

Therefore it cancels out, and it ends up being, on average, constant time. Just with a larger constant to account for occasional long copies.

Wait...so amortized time is average time?

In this particular case, yes. The amortized time is the same as average time. That's because we're only considering one operation: append.

But what about with trees?

Amortized analysis for trees

Trees need to be careful. They are used for dictionaries. Suppose we're using the dictionary to store user activity because we want to see all the activity over the previous month or something. We have to be very careful.

A user might realize that we're using a tree, and craft some activity that hits the worst case for the tree with the goal of slowing down our server. This is called a denial of service attack.

Amortized analysis is when you consider every possible action a user could take, and conclude that in every possible case, the total amount of time is bounded by some function of n .

Amortized analysis in general

Amortized analysis is more complicated than the tricks we've used so far.

There's typically some kind of accounting that needs to be done.

One way of doing it is to define a relation that represents all the valid states our data structure could be in, and then do induction over derivations of the relation.

This kind of analysis features in Ph.D. level coursework on algorithms and it's outside the scope of this class.

However, we can use the amortized analysis of others...

Amortized analysis for scapegoat tree sizes

When we're computing the size of each ancestor in the scapegoat tree, we will touch at most n nodes, where n is the size of the scapegoat.

Then we will re-build the scapegoat, which is also $O(n)$

However, both of these operations occur fairly rarely. Even in the worst case scenario, with $.5 < \alpha < 1$, the insertion itself will be $O(\lg n)$, and then some time will pass before we have to do it again. As the tree gets big, even with alpha very close to .5, this holds.

Rebuilding the tree

We've talked about how to insert, how to compute the sizes of each ancestor, and how to find the scapegoat. We've also mentioned rebuilding the tree. How do you do it?

The easiest way: the tree is already in order because it is a binary search tree. Perform an in-order traversal on the scapegoat tree and write it into an array.

Use that array to rebuild the tree. Take the median as the new root. Now run the rebuild algorithm on the left and right partitions of that array, and make them the left and right children of the root.

This takes $O(n)$ time. The authors in the studies have a more complex re-build algorithm that avoid re-allocations, and involves manipulating the child node pointers of the scapegoat.

Questions?

Your assignment

Your next project involves using scapegoat trees.

Let's take a look!

Appendix

The shortest paper on scapegoat trees from the authors are [these ACM-SIAM proceedings](#).

They have enough information for you to implement scapegoat trees.

However, feel free to reach out for help if you get stuck.